ANALYSE DE FIRMWARE · ÉLECTRONIQUE SOLAIRE

Rétro-ingénierie du firmware d'un onduleur hybride

Démarche d'analyse, outillage (Claude + Ghidra), et cartographie des registres Modbus — cas du MCU de puissance Deye SUN (SG_6K · V4390)

TI C28x DSP · 466 fonctions · bus Modbus RTU / RS485 · matériel possédé par l'analyste

La démarche en bref

Transformer un binaire brut d'onduleur en une carte exploitable de son interface Modbus

01 Identifier la cible

Architecture, cœur, base mémoire — avant toute chose, savoir ce qu'on lit.

02 Charger correctement

Bonne base, bon adressage, périphériques et RAM mappés.

03 Trouver l'interface

Localiser le sous-système Modbus (UART/SCI, CRC, ISR).

04 Cartographier

Du dispatch de registres aux plages lecture/écriture réelles.

05 Valider & rester honnête

Séparer le prouvé du probable ; recouper avec la doc.

Pourquoi analyser un firmware d'onduleur

Interopérabilité

Piloter et superviser l'onduleur depuis un système tiers (domotique, ESP32) via Modbus.

Registres non documentés

La doc constructeur et l'application ne couvrent pas tout l'espace de registres réellement servi.

Surface d'accès & sécurité

Identifier commandes et accès non standard sur un bus RS485 exposé.

Compréhension physique

Relier les registres aux grandeurs réelles (courants, tensions, calibration).

Cadre de l'analyse

Matériel possédé par l'analyste.
Recherche personnelle, à des fins d'interopérabilité sur son propre équipement.

Binaire = **image flash brute** (non chiffrée) du MCU de puissance.

Objectif : **cartographier l'interface Modbus**, pas modifier l'onduleur.

Méthodologie générale d'analyse d'un firmware

Un pipeline reproductible, du binaire inconnu à l'interprétation fonctionnelle

1 Acquisition

Dump / fichier binaire, hypothèses sur le format

2 Reconnaissance

Entropie, chaînes, byte-order, recherche d'architecture

3 Chargement

Base d'adresses, mapping mémoire, périphériques & RAM

4 Désassemblage

Désassembleur + décompilation, repérage de _c_int00 / main

5 Analyse fonctionnelle

Suivi des sous-systèmes (série, CRC, dispatch registres)

6 Validation

Recoupement avec la doc, distinction prouvé / probable

PARTIE 2

L'association Claude + Ghidra

Un décompilateur piloté par un assistant, et pourquoi REA n'a pas convenu

Claude + Ghidra : une boucle d'analyse

G — Ghidra

Le moteur

- Désassemblage & décompilation
- Chargement du module C28x
- Scripts headless (Java) rejouables
- Base du projet, références croisées

C — Claude

Le pilote

- Génère & lance les scripts headless
- Lit le décompilé, formule des hypothèses
- Suit les sous-systèmes de proche en proche
- Rédige notes, livrables et recoupe la doc

A — Analyse

Le résultat

- Carte mémoire & fonctions nommées
- Sous-système Modbus identifié
- Plages de registres R/W
- Livrable PDF des registres

Itération : chaque hypothèse de Claude se vérifie par un script Ghidra, dont le résultat nourrit l'hypothèse suivante.

Le rôle de chaque outil — et la limite de REA

Ghidra

Suite de rétro-ingénierie (NSA)

RETENU

Décompilateur ouvert, scriptable en headless. Module tiers ajouté pour le cœur TI C28x. Base de tout le travail.

Claude

Assistant d'analyse

RETENU

Orchestre Ghidra, interprète le décompilé, tient le fil des sous-systèmes et produit les livrables.

REA

« Reverse Engineer Anything »

ÉCARTÉ

Projet de rétro-ingénierie très populaire (clones de produits Adobe, etc.). Mais il ne prend pas en charge l'architecture TI C28x de ce MCU — inadapté à cette cible, abandonné pour ce firmware.

PARTIE 3

Les étapes clés du cas Deye

De l'identification de l'architecture à la cartographie complète des registres Modbus

Étape 1 — identifier l'architecture

Le préfixe « SG_6K » est trompeur : ce n'est pas un Sungrow.

Cœur = TI C2000 / C28x (famille Delfino F2833x, typiquement un F28335).

Image brute, non chiffrée (entropie 6,4) et non byte-swappée — chargeable telle quelle.

Mémoire adressée par mot : 1 adresse = 16 bits.

0x300000

Base d'adresses confirmée

370 cibles d'appel tombent dans l'image, 0 flux EMIF externe

0x31544b

_c_int00 (point d'entrée)

main à 0x30001e · données init par .cinit inline

6,4

Entropie de l'image

→ code clair, non chiffré

Fiche technique du MCU — TI C28x (Delfino F2833x)

Caractéristique (TI C28x — Delfino F2833x / F28335)	Utilisé par le firmware
Cœur CPU C28x 32 bits, ~150 MHz	Oui — contrôle temps réel
FPU virgule flottante 32 bits	Oui — instructions MPYF32 présentes
Flash interne (base 0x300000, mot de 16 bits)	Oui — image ≈ 230 Ko
RAM interne M0/M1 + SARAM L0–L7 + H0	Oui — données / buffers
ePWM (jusqu'à 18 sorties)	Oui — ePWM 1-6 (conversion de puissance)
ADC 12 bits (16 voies)	Oui — mesures courant / tension
GPIO (jusqu'à 88)	Oui — relais, LED, E/S
SCI-A (UART)	Oui — Modbus RTU / RS485 (ESP32 / logger externe)
SCI-B (UART)	Oui — console / calibration (debug usine)
SCI-C (UART)	Oui — lien inter-MCU (carte comms/HMI)
eCAN-B (bus CAN)	Oui — BMS batterie
eCAN-A (bus CAN)	Non — non initialisé
I2C-A	Oui — EEPROM de config 0x51 (24Cxx)
SPI-A	Non — non initialisé
McBSP-A / McBSP-B	Non — non initialisés
eCAP / eQEP	Non — non utilisés
Watchdog	Oui — + reset logiciel (WDCR)
USB	Absent du silicium C28x
Radio / modem cellulaire (4G…)	Non — aucun : pas de 4G sur ce MCU

Étape 2 — charger et mapper la mémoire

Module processeur

Extension C28x tierce compilée et installée dans Ghidra — langage TMS320C28x (FPU conservé).

Carte mémoire

Périphériques, RAM M0/M1, blocs L0–L7 (SARAM) et H0 mappés à la main (scripts Setup).

Point d'entrée

_c_int00 → main localisés ; initialisation des données par .cinit inline (pas de copy-table).

Résultat du chargement

466 fonctions · 99,1 % couvertes (appels + entrées async/ISR) · 0 fonction inexpliquée · indéfini restant = données (.cinit, tables const, flash vierge).

Étape 3 — localiser le sous-système Modbus

Protocole = Modbus RTU sur liaisons série SCI (RS485)

Les ports série

SCI-A = Modbus RTU, 9600 ou 57600 bps, 8N1 (port ESP32 / RS485)
SCI-B = console/calibration, 9600·4800
SCI-C = lien MCU1↔MCU2 (0xA2/0xAB)
· eCAN-B = BMS (CAN séparé)

Débit SCI-A = écrire 9600 ou 57600 au registre 45 (0x2d).

Repères trouvés

CRC16/Modbus (poly 0xA001, init 0xFFFF)
ISR de réception → tampons circulaires en RAM
Construction de trames FC 0x03 / 0x10

Les dispatchers

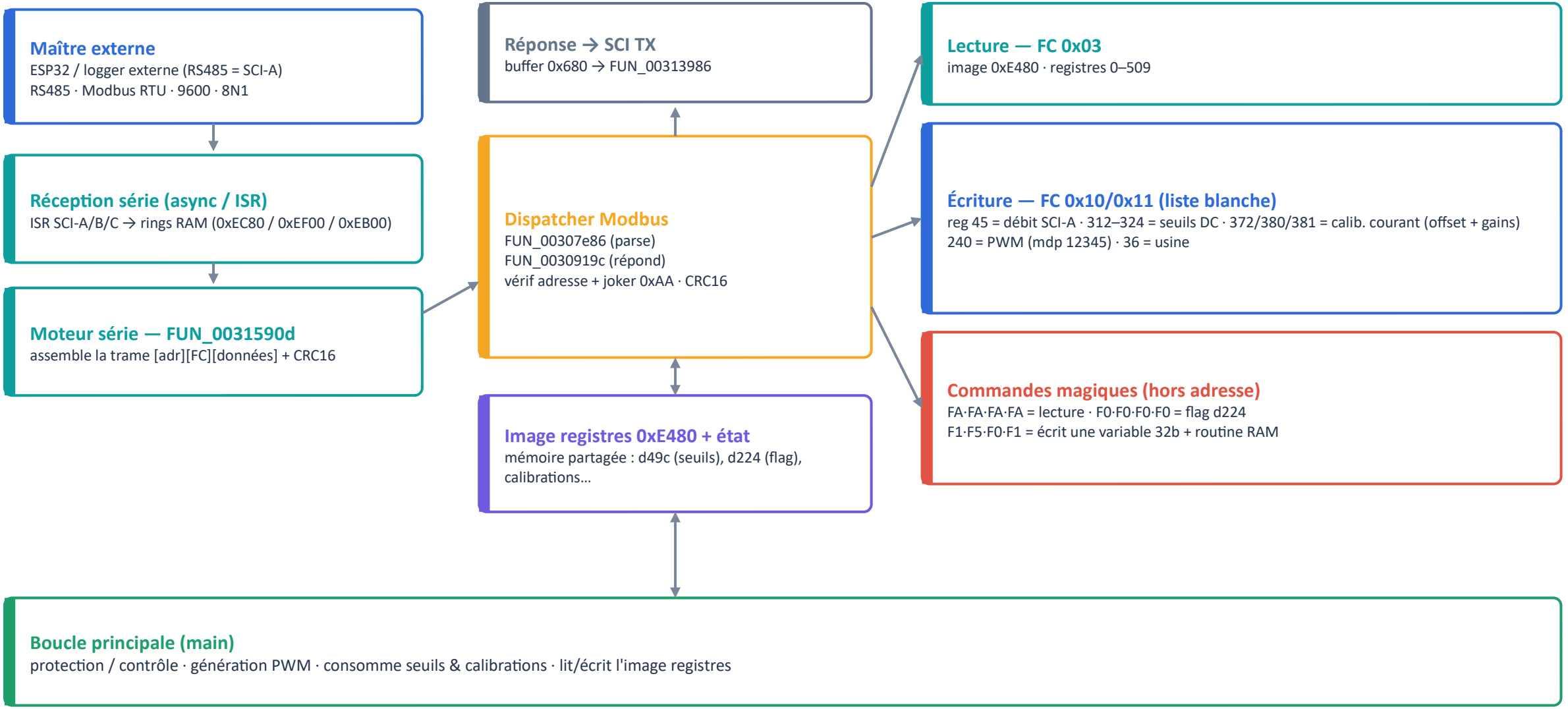
Deux gestionnaires esclave volumineux parsent la trame [adresse][fonction][données] et aiguillent selon le n° de registre reçu.

Changer le débit de SCI-A — Modbus RTU, FC 0x10, écriture du registre 45 (0x2d) :

9600 bps `01 10 00 2D 00 01 02 25 80 BB 1D` **57600 bps** `01 10 00 2D 00 01 02 E1 00 E8 7D`

Adresse « joker » 0xAA au lieu de 01 → CRC recalculé (9600 : …30 2A · 57600 : …63 4A).

Vue d'ensemble — du RS485 aux handlers et à la boucle de contrôle



Étape 4 — cartographier tous les registres

Comment les registres sont servis

Image registres à plat en RAM (base 0xe480) : registre N = mot 0xe480 + N.

Lecture (FC 0x03) : copie à plat, bornée → registres 0–509 lisibles.

Écriture (FC 0x10/0x11) : liste blanche stricte de plages ; hors bornes = refusé.

Certains registres déclenchent une action ; d'autres sont persistés en NVM.

La méthode : recoupement systématique

- Doc officielle V119 (monophasé 2022)
- Doc triphasé V105.4 (2024, plus large)
- Catalogue de l'application Deye (161 registres)
- Registres réellement servis par le firmware

→ Diff des trois sources : tout registre servi par le firmware mais absent des docs = candidat « caché ».

Ce que la cartographie a révélé

Registres cachés

439–499 · 312+

439–499 : config/NVM lisible + inscriptible, hors doc
125–149 : servis par le FW (seul 142 dans l'appli)
312–324 : seuils tension DC ~400–588 V (protection)
→ côté bus DC / PV, pas la batterie (unité LV)

Calibration des courants DC

offset + gains

Modèle affine : mesure = gain × brut + offset
Gains 380 / 381 : clamp [900,1100] (±10 %)
Offset 372 : clamp [0,2000] (±1000 autour de 0)
→ voie DC recombinée = MPPT / bus DC, pas batterie

Commandes hors espace registres

FA·F0·F1

FA FA FA FA → lecture d'une valeur 32 bits
F0 F0 F0 F0 → lecture + pose d'un flag (arm)
F1 F5 F0 F1 → écriture d'une variable 32 bits
Reconnues SANS vérif d'adresse Modbus

Accès & protections

0xAA · 0xF0

0xAA : adresse ESCLAVE « joker » (champ adresse de la trame)
reg 240 (= 0x00F0) : registre de test usine PWM, mdp 12345
Écriture bornée par une liste blanche de plages

Zoom — calibration affine des courants (gain + offset)

mesure = gain × brut + offset — trois registres, deux rôles ; validation en live pour lever les derniers inconnus

Le modèle

372 — OFFSET

0x174 · clamp [0,2000] → d4e4 = valeur − 1000 (±1000 centré sur 0). Ajouté à une voie courant interne (seuils de protection).

380 — GAIN

0x17C · clamp [900,1100] (±10 %). Courant direct à source unique → exposé reg 129.

381 — GAIN

0x17D · clamp [900,1100]. Courant recombiné multi-strings, forcé en mode batterie → exposé reg 128/145.

Offsets internes en plus (non pilotables ici) : zéro ADC figé (brut − 2042) + d7c2/d842/d346.

Trames de test — FC06, appareil 0x01

gain 380 = 1100 (+10 %)

```
01 06 01 7C 04 4C 4A DB
```

gain 381 = 1100 (+10 %)

```
01 06 01 7D 04 4C 1B 1B
```

offset 372 = 1500 (+500)

```
01 06 01 74 05 DC CA E5
```

offset 372 = 500 (−500)

```
01 06 01 74 01 F4 C8 3B
```

lire courants 128–149

```
01 03 00 80 00 16 C5 EC
```

Gain → effet proportionnel au courant. **Offset** → à courant ≈ 0, décalage constant. Toujours revenir à 1000 (neutre).

Calibration courants — tracé firmware du gain et de l'offset

Deux gains réglables (380/381)

× gain

Deux coefficients de GAIN du courant (registres 380 et 381), bornés [900,1100] — 1000 = neutre, soit ±10 %. Le firmware calcule : courant = gain × mesure brute. (vars 0xD4E1/0xD4E2, fonction FUN_0030c826.)

Un offset réglable (372)

+ offset

Décalage ajouté APRÈS le gain, sur la MÊME voie de courant que le gain 0xD4E1 (donc pas indépendant). Borné [0,2000] — 1000 = zéro. Modèle complet : courant = gain × brut + offset. (var 0xD4E4.)

Une voie DC recombinée

somme

Les courants calibrés sont ADDITIONNÉS à d'autres voies (0xDA80 + 0xDA82 + …) → un courant DC agrégé, PAS une mesure isolée ni le courant batterie.

Plutôt MPPT / bus DC

≈ MPPT

Cette voie recombinée penche côté MPPT / bus DC général (pas la batterie, unité LV). Le n° de registre Modbus exact n'est pas lisible en statique → confirmer en live : écrire un gain = 1100 et voir quelle grandeur bouge.

Honnêteté d'analyse : prouvé vs probable

✓ Établi (solide)

- Architecture C28x, base 0x300000, 466 fonctions
- Les ports SCI et leur rôle respectif
- Plages de lecture/écriture réelles des registres
- Registres cachés 439–499 (recoupés sur 3 sources)
- Commandes magiques décodées : FA·FA·FA·FA = lecture, F0·F0·F0·F0 = pose d'un flag, F1·F5·F0·F1 = écriture d'une variable 32 bits + réponse
- Reconnues sans contrôle d'adresse (comparaison octet par octet)
- Config flash = wait-states de lecture seulement (InitFlash)

! Rétracté / non prouvé

« Reflash par RS485 » : non étayé
F1F5F0F1 écrit une variable RAM et formate un nombre (routine appelée = un itoa ; l'autre, résidente RAM, est un helper générique appelé 116× au boot). Aucune écriture du tableau flash, aucun accès au mot de passe CSM.

Reset / redémarrage distant — depuis LOCALISÉ
N'était aucune des 3 commandes magiques. DEPUIS LOCALISÉ : reboot watchdog (WDCR=0) via la trame 0xAB sur SCI-C ; clear arc-fault = reg 181. Voir slide « redémarrage distant ».

Leçon : *un scan de motifs ne prouve pas une absence ; recouper avec le décodage des handlers.*

Ce qui reste à faire

Table registre → grandeur physique

Priorité

Relier chaque n° de registre à la mesure réelle (ADC, télémétrie). Gros travail, cœur de l'interprétation.

Validation en conditions réelles

Terrain

Calibration courants : gains 380/381 (effet proportionnel au courant) et offset 372 (décalage constant à courant nul) — écrire puis observer quelle voie bouge (lève aussi l'ambiguïté batterie vs MPPT).

Reset d'alarmes / redémarrage distant — RÉSOLU

Fait

Trouvé : reboot watchdog via trame 0xAB (SCI-C / lien MCU1) → WDCR=0 @0x8F830 ; clear du défaut d'arc = reg 181 (mdp 12345). Reste à confirmer en live les flags défaut/alarme R/W 220/221/222.

Dongle & comms (firmware chiffré)

Bloqué

Dongle WiFi Hi-Flying + carte MCU1 : firmwares chiffrés (AES, clé introuvable sans JTAG). Relaie le cloud Deye et transmet au MCU2 via SCI-C (trames 0xAB) — d'où le reboot distant. Reflash : non exposé côté MCU2. Déchiffrer MCU1 = bloqué sans JTAG.

Mise à jour — le redémarrage distant (via le dongle)

Déclencheur

0xAB

Trame propriétaire 0xAB sur le lien inter-MCU, parsée par FUN_0031590d (même voie et même CRC-16 que le Modbus). Émise par le dongle / MCU1 — incapturable en local : seul Deye, à distance via le dongle, peut l'émettre.

Reset du MCU2 (bien réel)

WDCR = 0

La trame pose un flag RAM 0xD169 ; la boucle principale exécute alors EALLOW ; WDCR=0 @0x8F830 → WDCHK illégal → reset watchdog = reboot du MCU de puissance. (Variante WDCR=0x3F présente mais code mort.)

Structure de la trame entrante

reg 0x0438

AB 10 04 38 00 02 D9 3F — AB=adresse, 10=FC16 (write), 04 38=registre 0x0438, 00 02=quantité 2 reg, D9 3F=CRC. Le reset est identifié par la COMMANDE (reg 0x0438, qté 2), pas par une valeur. Ces 8 octets = forme réponse/écho ; requête réelle = AB [FC] 04 38 00 02 …

Clear d'alarme (arc-fault)

reg 181

Écriture du registre 181 = 2 (mot de passe 12345) : machine d'état qui s'auto-réaligne (doc : repasse à 01). Efface le défaut d'arc — ce n'est PAS un reboot. Flags défaut/alarme R/W 220/221/222 aussi.

Pas de voie cellulaire sur le MCU de puissance

Aucune pile réseau

0 trace

Recherche exhaustive dans l'image (230 Ko) : aucune commande AT, ni CSQ/CREG/PPP/GPRS/LTE/MQTT/TCP/HTTP/APN/SIM/modem/dongle/wifi. Un modem se pilote en texte (AT+…) → leur absence est concluante. Et le C28x n'a pas d'USB.

3 UART tous occupés

SCI-A/B/C

SCI-A = Modbus/RS485 (ESP32 + dongle), SCI-B = console/calibration, SCI-C = 57600 fixe. Aucun ne porte de protocole modem. Le seul vecteur 4G réaliste serait un UART libre — il n'y en a aucun.

Bus réellement utilisés

eCAN-B + I2C

eCAN-B = BMS (292 accès). I2C-A = EEPROM de config 0x51 (adressage 16 bits, 36 accès). ePWM, GPIO, ADC = conversion de puissance. Rien d'orienté réseau.

Bus libres → conclusion

SPI-A inerte

SPI-A, McBSP-A/B, eCAN-A, eCAP/eQEP = NON initialisés : aucune interface candidate pour un modem externe. ⇒ backdoor cellulaire au niveau MCU2 = ÉLIMINÉE. Le lien vers le cloud Deye est dans le DONGLE (processeur séparé).

Architecture : où vit l'écran et comment il lit les données

MCU2 (puissance)

serveur

C28x. Convertit la puissance, mesure (ADC) et EXPOSE ses données comme un serveur Modbus/vendor. Ne pilote AUCUN écran : SPI inerte, I2C = EEPROM. Il répond, il n'affiche pas.

MCU1 (carte comms / HMI)

client

Processeur séparé (firmware chiffré). Interroge le MCU2, PILOTE l'écran localement, gère le dongle WiFi/4G et le cloud Deye. C'est lui qui « voit » le réseau.

Le lien MCU1 ↔ MCU2

SCI-C

Liaison série SCI-C (0x7770, 57600 fixe) : trames propriétaires 0xA2/0xAB + télémétrie 0x68 reçues dans le tampon 0xeb00, traitées par le moteur série FUN_0031590d. Même CRC-16 Modbus.

L'écran lit en Modbus ?

indirect

Oui mais INDIRECTEMENT : les chiffres voyagent MCU2→MCU1 en Modbus/vendor sur SCI-C ; l'afficheur physique est câblé sur MCU1. (SCI-A = RS485 externe ESP32/logger ; SCI-B = console calibration.)

Console SCI-B — paramètres de régulation (calibration usine)

Nature

INTERNE

Port d'USINE sur la carte de contrôle, NON exposé à l'extérieur : aucun connecteur accessible (le RS232/WiFi en bas = SCI-A, pas SCI-B) → accès = OUVRIR l'appareil. Console TEXTE 9600/4800 8N1 : chaque token d'une ligne (0xef00) est lu, borné, rangé.

Syntaxe

<TOKEN><val>

Format : <TOKEN><valeur> en ASCII, 9600 8N1, SANS CRC. Ex. régler le Kp MPPT (PV_KP) à 1000 → envoyer « PV_KP1000 » = octets 50 56 5F 4B 50 31 30 30 30 ; écho « +PV_KP1000 ». Valeurs bornées ; noms @0x31b9d0.

Boucles PI réglables

dq / P / batt

Gains kp/ki : courant dq (Id/Iq), PI bus, tension réseau (Un), batterie charge/décharge (BC/BD/BS), MPPT PV (PV_KP/KI/MP), puissance (PW_P1..P9 / PW_i1..i9). = cœur du contrôle de l'onduleur.

Commandes & prudence

usine

Autres tokens : start, TE_ST, Se_tP, AL_SE, DA_TA, ADCEL, PO_OW/O2/O3… Console de réglage USINE : des gains erronés peuvent déstabiliser la régulation. Liste complète : sci_b_console_calibration.md.

Modèle d'onduleur : fixé par un code en EEPROM (bridage)

Descripteur EEPROM

0xEE12/14/16

Au boot, un bloc est lu de l'EEPROM 0x51 (FUN_0030b367 → FUN_00314eb3) : 0xEE12 = signature/type, 0xEE14 = code MODÈLE + options (12 bits bas : 0x514/0x516/0x525…), 0xEE16 = puissance nominale. Device Type (reg 0) = constante 0x200 figée.

Firmware générique

branches

Un seul binaire pour plusieurs modèles : le code 0xEE14 est testé PARTOUT — calibration (FUN_0030c826), protections/limites (FUN_0030cfe3, 0030ea3a), options, format télémétrie. Chaque modèle a son jeu de coefficients câblé, choisi par le code.

Limites = code, pas le nombre

0xEE14

Les plafonds courant/puissance et la calibration dérivent du CODE MODÈLE (0xEE14), PAS de la valeur « puissance nominale » (0xEE16, surtout reportée + setter runtime). Relever seul 0xEE16 change l'affichage, pas le comportement réel.

Bridage, mais borné

≠ unlock

C'est bien une sélection logicielle/EEPROM (modèles frères sur une base commune). MAIS : coefficients limités à ceux que le firmware connaît ; capacité réelle bornée par le MATÉRIEL ; EEPROM probablement checksummée + accès I2C physique. Pas un unlock trivial ni sûr.

EN RÉSUMÉ

Un binaire brut devenu une carte Modbus exploitable

466	0x300000	~128	439–499
fonctions analysées	base confirmée	registres cartographiés	registres cachés isolés

Méthode reproductible · outillage Claude + Ghidra · prochaine étape : relier chaque registre à sa grandeur physique.